

Multi-Forge: uma arquitetura declarativa, reproduzível e tolerante a falhas para reaproveitamento de TV Boxes ARM64 como nós de computação de borda

Brenda Biral

Adriel Henrique Souza

Marcos Oliveira e Silva

Isaac Andrade

Instituto de Ciência e Tecnologia de Sorocaba (ICTS), UNESP

1º Hackathon TV Box UNESP Sorocaba, Equipe 1

Versão técnica baseada no repositório em 17 de setembro de 2026

Resumo

O descarte crescente de equipamentos eletrônicos e a demanda por capacidade computacional próxima ao usuário motivam arquiteturas capazes de reaproveitar hardware de consumo como infraestrutura de borda. Este artigo apresenta a MULTI-FORGE, uma plataforma de software concebida para transformar uma TV Box BTV Express E10, originalmente destinada a consumo multimídia, em um appliance ARM64 provisionável, recuperável e extensível. A plataforma combina: (i) uma distribuição Linux derivada de Armbian, denominada FORGEOS, com árvore de dispositivos específica para o Amlogic S905X2; (ii) um pipeline de construção de imagem que injeta provisionamento, serviços e parâmetros de memória e rede; (iii) o FORGEIMAGER, gravador desktop em Rust/Tauri capaz de consumir um catálogo declarativo e injetar configuração em ext4; (iv) o FORGEDB, catálogo versionado e validado por JSON Schema; (v) o FORGEHUB, daemon de gerenciamento em Go com telemetria, proxy dinâmico e controle de módulos; e (vi) uma pilha de provisionamento sem periféricos, baseada em ponto de acesso Wi-Fi, portal local, HDMI/framebuffer e rollback automático.

A contribuição técnica central é uma separação explícita entre descrição declarativa do hardware, produção da imagem, bootstrap de rede, gerenciamento em tempo de execução e aplicações. O estudo utiliza como caso de referência a BTV Express E10, equipada com Amlogic S905X2, quatro núcleos Cortex-A53, 2 GB de RAM e 8 GB de eMMC. A imagem FORGEOS v2.2.0 publicada possui 738.523.064 bytes comprimidos e SHA-256 publicado. O artigo também realiza uma auditoria de evidência do artefato: mecanismos efetivamente injetados pela imagem, como ZRAM configurada para 50% da RAM, parâmetros `sysctl`, ajustes do RTL8189FTV e DTB com CMA de 64 MB, são separados de opções de kernel descritas no repositório mas não compiladas pelo pipeline atual. Essa distinção evita atribuir ganhos de desempenho não medidos. Finalmente, discutimos reprodutibilidade, segurança, ameaças à validade e um protocolo experimental para validação quantitativa futura de tempo de boot, memória, estabilidade, temperatura, energia e recuperação de falhas.

Palavras-chave: edge computing; Linux embarcado;

ARM64; reutilização de hardware; TV Box; provisionamento; sistemas declarativos; computação sustentável.

Abstract

The growth of electronic waste and the demand for computation closer to users motivate architectures capable of repurposing consumer hardware as edge infrastructure. This paper presents MULTI-FORGE, a software platform designed to transform a BTV Express E10 TV box into a provisionable, recoverable, and extensible ARM64 appliance. The platform combines an Armbian-derived Linux image (FORGEOS), a hardware-specific device tree, an image construction pipeline, a Rust/Tauri desktop imager, a declarative hardware and image catalog, a Go-based edge management daemon, and a peripheral-free Wi-Fi/HDMI provisioning stack. The main architectural contribution is the separation of hardware description, image production, network bootstrap, runtime management, and application modules. We additionally perform an evidence audit of the repository, distinguishing mechanisms directly applied by the image builder from kernel options merely documented as targets. This prevents unmeasured optimization claims from being presented as experimental results. The paper concludes with reproducibility and security limitations and a quantitative evaluation protocol for future work.

1 Introdução

A computação de borda desloca processamento, armazenamento e decisões para regiões próximas às fontes de dados e aos usuários, reduzindo dependência de datacenters remotos em aplicações sensíveis a latência, disponibilidade e privacidade [33, 36]. A ideia também aparece na literatura de fog computing, que posiciona recursos intermediários entre dispositivos finais e a nuvem [1]. Paralelamente, a produção mundial de resíduos eletrônicos atingiu 62 bilhões de quilogramas em 2022, dos quais apenas 22,3% foram documentados como formalmente coletados e reciclados [29]. Esses dois problemas sugerem uma oportunidade: prolongar a vida útil de dispositivos capazes de executar Linux e convertê-los em plataformas

de borda.

TV Boxes ARM constituem um caso particularmente interessante. Em geral, oferecem SoCs multicore, interfaces HDMI, USB, Ethernet, Wi-Fi, armazenamento flash e consumo energético inferior ao de computadores convencionais, mas apresentam heterogeneidade de bootloader, árvores de dispositivos, periféricos e drivers. Essa heterogeneidade torna a simples instalação de uma distribuição Linux insuficiente para produzir um appliance confiável.

A MULTI-FORGE foi desenvolvida nesse contexto, inicialmente para o reaproveitamento da BTV Express E10 no 1º Hackathon TV Box UNESP Sorocaba. O dispositivo de referência possui Amlogic S905X2 (família Meson G12A), quatro núcleos Cortex-A53, 2 GB de LPDDR4, 8 GB de eMMC e rádio Realtek RTL8189FTV conectado por SDIO [7]. A plataforma não é apenas uma imagem Linux. Ela organiza o ciclo de vida completo do dispositivo: identificação do hardware, seleção da imagem, gravação, bootstrap de rede, recuperação de conectividade, telemetria, execução de aplicações e distribuição declarativa de metadados [8, 16, 25].

Este artigo tem três objetivos. Primeiro, documentar de forma científica a arquitetura atual da MULTI-FORGE e da imagem FORGEOS. Segundo, sistematizar as otimizações de construção, memória, rede, boot, framebuffer e árvore de dispositivos aplicadas ao hardware de referência. Terceiro, auditar o grau de evidência de cada otimização. Essa última etapa é essencial porque um repositório em evolução mistura código executado, configuração pretendida, documentação histórica e metas de projeto. Em sistemas, confundir esses níveis produz conclusões impossíveis de reproduzir.

As contribuições deste trabalho são:

1. uma arquitetura em camadas para reaproveitamento de TV Boxes ARM64 como appliances de borda;
2. um modelo declarativo de dispositivos, imagens e fingerprints por meio do FORGEDB;
3. um pipeline automatizado de construção e validação de imagens que integra DTB, provisionamento, serviços e checksum;
4. um mecanismo de onboarding sem teclado ou mouse, baseado em AP local, portal e HDMI;
5. uma arquitetura de execução modular com guarda de memória, systemd/Compose, proxy dinâmico e telemetria;
6. uma auditoria de evidência que separa otimizações *aplicadas, configuradas, documentadas e ainda não demonstradas experimentalmente*;
7. um protocolo de avaliação reprodutível para medir desempenho e estabilidade em hardware real.

2 Escopo, metodologia e snapshot do artefato

A análise foi realizada sobre a branch `main` do repositório público <https://github.com/multi-forge/multi-forge>, no commit `f670eb822fb40d6fda17683496c85b04b2711dec`, observado em 17 de setembro de 2026 [26]. A release de produção mais recente analisada foi a FORGEOS v2.2.0, publicada na mesma data, com imagem

Tabela 1: Hardware de referência segundo o descritor `device.yaml`.

Componente	Especificação
SoC	Amlogic S905X2, família Meson G12A
CPU	4× ARM Cortex-A53, até 1,8 GHz
Arquitetura	ARM64
RAM	2 GB LPDDR4
Armazenamento	8 GB eMMC
Wi-Fi	Realtek RTL8189FTV, barramento SDIO
USB	1× USB 2.0 e 1× USB 3.0
Vídeo	HDMI
Rede cabeada	Ethernet
Boot	SD/eMMC; DTB específico; fallback SEI510
Recuperação	MaskROM USB declarada no catálogo

comprimida de 738.523.064 bytes e digest SHA-256 `e452a149a0b871aac165d737b61a939a216734668365f977975567` [21].

A metodologia de auditoria classifica cada afirmação em quatro níveis:

E1 – implementado e aplicado: há código no pipeline atual que injeta ou habilita explicitamente o mecanismo na imagem;

E2 – implementado no runtime: há código executável no repositório, mas sua presença final depende do processo de empacotamento ou de artefatos externos;

E3 – declarado/configurado: existe configuração ou documentação que expressa a intenção, porém o pipeline observado não demonstra que o mecanismo foi incorporado;

E4 – alegado/experimentalmente pendente: há afirmação de ganho quantitativo sem dados brutos suficientes no snapshot analisado.

Essa classificação é usada ao longo do texto. Ela não implica que um mecanismo E3 esteja ausente do dispositivo real; apenas significa que sua ativação não pode ser deduzida exclusivamente do pipeline versionado.

3 Plataforma de hardware

A especificação declarativa do FORGEDB descreve a BTV Express E10 como TV Box ARM64 com as características apresentadas na Tabela 1 [7]. O mesmo descritor inclui fingerprints de CPU, Device Tree, USB, modelo de armazenamento e marcações de PCB, permitindo que clientes tentem reconhecer o hardware sem codificar regras específicas em cada aplicação.

A escolha desse hardware impõe restrições que orientam todo o desenho do software: memória pequena para aplicações contemporâneas, armazenamento flash limitado, rádio Wi-Fi de baixo custo, ausência de teclado/mouse no uso típico e necessidade de exibir estado em uma televisão. Consequentemente, a arquitetura favorece componentes pequenos, execução local, recuperação automática e redução de dependências.

4 Arquitetura geral da Multi-Forge

A Figura 1 sintetiza a arquitetura. A MULTI-FORGE separa o sistema em seis blocos principais.

4.1 ForgeCore e ForgeOS

O ForgeCore contém o pipeline de construção da distribuição, o Device Tree específico, parâmetros de sistema, configuração do rádio Wi-Fi e um harness QEMU AArch64 [6,18]. A imagem atual parte de uma base Armbian 26.08/Trixie para S905X2 com kernel 6.18.44-ophub e a modifica offline por loop device e chroot [18].

4.2 ForgeDB

O FORGEDB transforma conhecimento sobre hardware em dados declarativos. Dispositivos, imagens e fabricantes são descritos em YAML e validados por JSON Schema Draft 2020-12. Um pipeline de CI compila o material para `catalog.json`; o cliente pode consumi-lo por CDN, GitHub Raw ou cache local [8]. No snapshot analisado, o catálogo compilado declara um dispositivo suportado e cinco imagens FORGEOS [26].

4.3 ForgeImager

O FORGEIMAGER é um aplicativo desktop em Rust/Tauri v2 e React 19. A ferramenta declara suporte a enumeração de discos específica por plataforma, streaming de imagens comprimidas, verificação SHA-256 e injeção userspace de arquivos em ext4 por meio do crate `forge-write-conf` [16]. A cascata de catálogo busca CDN, GitHub Pages, GitHub Raw, cache local e snapshot embutido. Dessa forma, seleção de dispositivos e imagens pode evoluir sem recompilar a interface.

4.4 ForgeProvisioner

O FORGEPROVISIONER fornece o caminho de primeiro boot: ponto de acesso autônomo, DNS/DHCP local, portal HTTP, configuração de cliente Wi-Fi, framebuffer/HDMI e watchdog [22]. A versão atual também possui uma implementação mais nova em `ForgeHub/hardware`, usada pela release 2.2.0 para AP, watchdog e kiosk [21].

4.5 ForgeHub

O FORGEHUB é um daemon Go. Seu ponto de entrada utiliza `chi`, armazena estado com `bbolt`, embute os assets web no binário, inicializa proxy dinâmico e expõe APIs de gerenciamento [10]. O runner suporta módulos do tipo `systemd` e `Docker Compose` e realiza uma verificação de memória disponível antes de iniciar aplicações; o limiar padrão é 300 MB [13]. O subsistema de telemetria usa `gopsutil` para CPU, RAM, disco e rede e lê o sensor térmico em `/sys/class/thermal/thermal_zone0/temp` quando disponível [15].

4.6 ForgeModules e aplicações

A especificação de módulos define metadados, requisitos de RAM e armazenamento, dependências, origem e comandos de ciclo de vida [17]. No snapshot da branch `main`, entretanto, o diretório `ForgeModules` contém apenas seu README; os diretórios de módulos descritos nessa documentação não estão presentes na árvore observada. Portanto, o modelo de manifesto é tratado neste artigo como interface arquitetural documentada, e não como evidência de que todos os módulos listados estejam reproduzíveis a partir do clone atual.

5 Engenharia da imagem Linux

5.1 Pipeline de construção

A Figura 2 apresenta o fluxo observado em `build-image.sh`. O processo não compila uma distribuição do zero. Ele baixa uma imagem Armbian S905X2 pré-construída, descompacta-a, mapeia partições, injeta componentes, executa customização via `chroot`, valida o boot em QEMU e comprime a imagem final [18].

O builder configura paralelismo com `MAKEFLAGS=-j$NPROC`, habilita diretório de `ccache`, utiliza `pigz` para descompressão `gzip` e `xz -T0 -4` para compressão final. Quando detecta mais de 12 GB de RAM no host, tenta montar o diretório de trabalho em `tmpfs` de 16 GB. Para execução remota, `gcp-spot-launcher.py` tenta instâncias Spot de 8 a 32 vCPUs, transfere o repositório, executa o builder, baixa os artefatos e destrói a VM ao término [23].

É importante interpretar essas técnicas corretamente. Como o pipeline atual predominantemente *reempacota* uma imagem preexistente, `ccache` e `MAKEFLAGS` não equivalem a uma compilação incremental do kernel. Os ganhos mais diretos do paralelismo estão na compressão/descompressão e em tarefas auxiliares. A avaliação de tempo de build deve, portanto, decompor download, I/O, `chroot`, QEMU e compressão, em vez de atribuir toda aceleração a “compilação paralela”.

5.2 Parâmetros de boot

O builder substitui `/boot/uEnv.txt` e define explicitamente o DTB da BTV E10. Entre os parâmetros observados estão `rootwait`, `rootfstype=ext4`, `fsck.fix=yes`, `fsck.repair=yes`, habilitação de `accounting` de memória/`cgroup`, vídeo HDMI 1080p60, desativação de `Plymouth`, `quiet` e `loglevel=3` [18]. Na v2.2.0, o boot silencioso e a manutenção de um TTY2 de emergência foram destacados como parte da estabilização do console [21].

O perfil atual também habilita SSH e, durante o build de bancada, configura login root e senha padrão. Isso é útil em desenvolvimento, mas constitui risco de segurança e não deve ser mantido em uma imagem institucional de produção. A Seção 15 discute essa limitação.

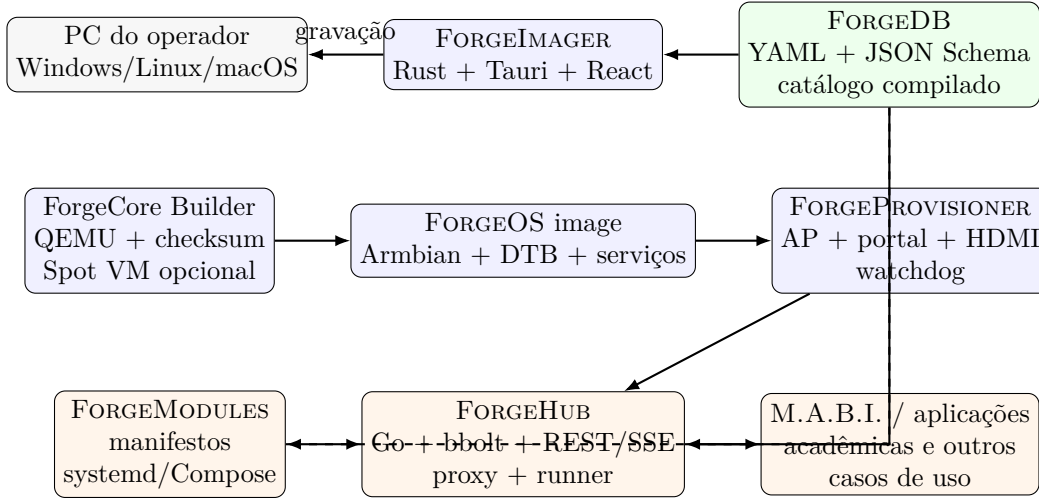


Figura 1: Arquitetura lógica da MULTI-FORGE. O catálogo declarativo desacopla identificação de hardware e imagens dos clientes; o ForgeCore produz a imagem; o Provisioner realiza bootstrap; e o ForgeHub gerencia o appliance em tempo de execução.

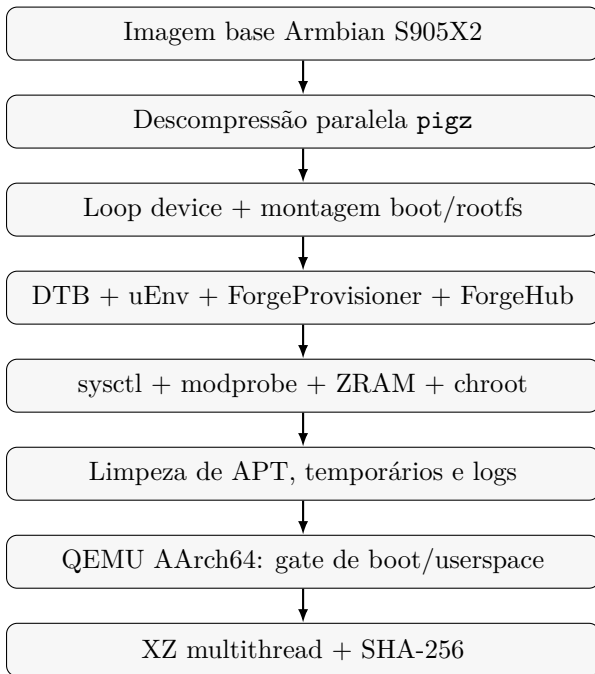


Figura 2: Pipeline de empacotamento da imagem FORGEOS.

5.3 Otimizações do Device Tree

O DTB `meson-g12a-btv-e10-enterprise` é uma das partes mais específicas do hardware. A Tabela 2 sumariza os ajustes documentados e versionados [4].

A redução de CMA é especialmente relevante em um sistema de 2 GB, mas o valor de “192 MB liberados” deve ser entendido como diferença de reserva configurada, não como aumento medido de `MemAvailable`. O impacto real depende de driver gráfico, uso do framebuffer e estado do sistema.

A limitação de SDIO a 25 MHz é apresentada pela documentação do projeto como medida de estabilidade para o RTL8189FTV. Uma avaliação científica completa deve reportar taxa de erro, desconexões e throughput em múltiplas frequências; o repositório atual demonstra a

Tabela 2: Alterações no Device Tree da BTV E10.

Ajuste	Descrição
CMA	Reserva <code>linux,cma</code> de 64 MB, em vez dos 256 MB indicados como base de referência. A diferença teórica é 192 MB.
SDIO	<code>max-frequency=25 MHz</code> e remoção de <code>high-speed</code> para o RTL8189FTV.
Wi-Fi	Compatibilidade explícita com RTL8189FS/FTV/generic-sdio.
Bluetooth	Nó de Bluetooth marcado como <code>disabled</code> por não existir no hardware alvo.
Watchdog	Nó Amlogic habilitado para disponibilizar <code>/dev/watchdog</code> .
Ethernet	Ajustes de atraso TX/RX no controlador Ethernet.
Identidade	<code>compatible = "btv,e10", ...</code> e modelo próprio.

configuração, mas não contém o experimento controlado correspondente.

5.4 Memória: ZRAM, pressão de cache e escrita

O builder cria `/etc/default/zramswap` com compressor ZSTD, tamanho de 50% da RAM física e prioridade 100 [18]. Em um dispositivo com aproximadamente 2 GB, isso configura um dispositivo de swap comprimido de ordem de 1 GB. A motivação é trocar CPU por menor pressão de RAM e evitar swap persistente em flash, um princípio consistente com mecanismos de memória comprimida do Linux [32]. Contudo, a taxa real de compressão depende da carga; valores como “3:1” não devem ser tratados como constantes.

O arquivo `99-forgeos-performance.conf` define:

```

vm.vfs_cache_pressure = 50
vm.dirty_background_ratio = 5
vm.dirty_ratio = 10
vm.swappiness = 60
vm.overcommit_memory = 1

```

e, para a rede:

```
net.core.default_qdisc = fq
net.ipv4.tcp_congestion_control = bbr
net.ipv4.tcp_fastopen = 3
net.ipv4.tcp_slow_start_after_idle = 0
```

além de buffers TCP e backlog [20].

Os limites menores de páginas sujas tendem a distribuir escrita ao longo do tempo e reduzir picos de flush em mídias lentas. A escolha é plausível para eMMC/MicroSD, mas seu efeito em latência e write amplification precisa ser medido.

5.5 MGLRU, BORE, escalonadores de I/O e aceleração criptográfica

O repositório contém `kernel_patches.config` com opções para MGLRU, ZRAM, flow offload, BORE, Kyber/BFQ, BBR, preempção, 300 Hz e criptografia ARMv8 [19]. MGLRU é uma implementação alternativa de reclaim do Linux voltada a melhorar a eficiência sob pressão de memória [31]. BBR, por sua vez, baseia o controle de congestionamento em estimativas de banda do gargalo e RTT, em contraste com algoritmos puramente orientados a perda [2].

Entretanto, o `build-image.sh` analisado baixa um kernel já compilado e **não invoca uma etapa de compilação do kernel que consuma `kernel_patches.config`**. Consequentemente:

- a presença do arquivo `Kconfig` é evidência de intenção E3;
- a ativação real deve ser verificada em `/proc/config.gz`, `sysfs` ou no binário do kernel;
- `tcp_congestion_control=bbr` seleciona a implementação `bbr` disponível, mas não prova, por si só, que ela corresponda à revisão “BBRv3”;
- BORE, BFQ/Kyber, MGLRU e crypto CE não devem ser anunciados como ativos sem inspeção do kernel de produção.

Essa distinção é uma das principais conclusões da auditoria técnica deste artigo.

5.6 Wi-Fi RTL8189FTV

O arquivo `8189fs-performance.conf` desativa gerenciamento agressivo de energia e ajusta parâmetros do driver:

```
options 8189fs rtw_power_mgnt=0
rtw_enusbss=0
rtw_hwpdn_mode=2
rtw_lowrate_two_xmit=1
```

[28]. Em conjunto com o clock SDIO de 25 MHz, o objetivo é privilegiar estabilidade do AP em vez de máximo throughput de barramento.

5.7 Serviços e redução de concorrência no framebuffer

A release 2.2.0 adota o princípio “um framebuffer, um writer, uma máquina de estados”: o serviço

`forge-kiosk.service` executa `kiosk_bridge.py`, enquanto serviços legados de display são mascarados [21]. A unit atual reserva TTY1, desativa cursor e blanking, reinicia o kiosk em caso de falha e aplica `OOMScoreAdjust=-500` [11]. Um drop-in substitui explicitamente `ExecStart` pelo bridge atual [14].

Esse desenho evita múltiplos processos escrevendo simultaneamente em `/dev/fb0`, um problema prático comum em appliances sem compositor gráfico. Ao dispensar X11/Wayland para a tela de provisionamento, a arquitetura reduz a quantidade de componentes no caminho crítico de boot. A aplicação M.A.B.I., quando executada com PyQt, constitui uma camada diferente de uso e deve ser tratada separadamente do kiosk mínimo.

6 Provisionamento sem periféricos e recuperação

6.1 Modo ponto de acesso

O controlador atual `forge-ap-ctrl` configura `wlan0` em 192.168.4.1/24, inicia `wpa_supplicant` em modo AP, inicializa `dnsmasq`, habilita encaminhamento IPv4 e cria regras NAT/redirect para o portal [5]. Antes disso, processos concorrentes de `hostapd`, `wpa_supplicant` e `dnsmasq` são encerrados e o endereço da interface é limpo.

O resultado é um bootstrap local que não depende de internet nem de periféricos de entrada. A televisão exibe QR code e estado, enquanto um telefone atua como interface de configuração.

6.2 Cliente Wi-Fi e redes institucionais

A arquitetura prevê transição de AP para cliente e inclui suporte documentado a WPA-Personal e WPA-Enterprise/802.1X. A release 2.2.0 destaca PEAP/MSCHAPv2/eduroam [21]. O uso institucional é importante porque um mecanismo restrito a WPA2-PSK não atenderia redes acadêmicas federadas.

6.3 Watchdog e rollback

O `watchdog` atual em `ForgeHub/hardware/network/watchdog.sh` verifica a interface a cada 5 s. Em modo cliente, considera a conexão válida quando existe IP distinto do AP e o gateway ou endereços externos respondem. Após 60 s contínuos de falha, reinicia o AP [12].

Há uma discrepância de documentação: a release v2.2.0 e o README citam 75 s, enquanto o script atual usa `TIMEOUT_SECS=60`. A implementação versionada prevalece na caracterização do artefato. Essa diferença deve ser eliminada antes de uma publicação final ou avaliação formal.

Uma implementação anterior em `FORGEPROVISIONER` adota política diferente: arquivo de estado `applying`, limite de 5 minutos para provisionamento travado e contagem de 20 falhas a intervalos de 15 s [24]. A coexistência de duas gerações de `watchdog` indica evolução arquitetural e reforça a necessidade de vincular resultados à versão exata da imagem.

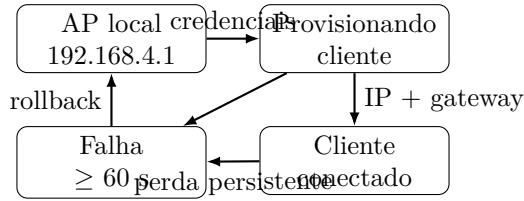


Figura 3: Máquina de estados simplificada de conectividade do runtime atual.

7 Catálogo declarativo e identificação de hardware

O FORGEDB desacopla suporte de hardware do código do imager. Um descritor contém:

- identidade, fabricante, categoria e estado de suporte;
- SoC, arquitetura, CPU, RAM, armazenamento e periféricos;
- fingerprints de `cpuinfo`, Device Tree, USB e armazenamento;
- parâmetros de boot, DTB primário/fallback e método de recuperação;
- problemas conhecidos e workarounds;
- catálogo de imagens, hashes e destinos de flash.

Essa abordagem é comparável, em espírito, a sistemas declarativos de deployment: a descrição do estado e das dependências se torna um artefato versionável, reduzindo decisões implícitas na aplicação [3]. O uso de schemas formais permite falhar no CI antes que um catálogo inválido chegue aos clientes.

O catálogo compilado observado no snapshot contém cinco imagens, com a v2.2.0 marcada como recomendada. O próprio catálogo registra versão de kernel, estabilidade, tamanho e URLs de download [26]. Essa cadeia é útil para provenance, mas ainda não produz builds bit-a-bit reproduzíveis. A literatura de reproducible builds distingue precisamente “conseguir reconstruir” de “obter binários idênticos a partir do mesmo fonte” [30]. O pipeline MULTI-FORGE atualmente está mais próximo de *build automation* e *artifact traceability* do que de reproducibilidade bit-a-bit estrita.

8 ForgeImager: aquisição, gravação e customização

O FORGEIMAGER usa o catálogo para selecionar imagens e identificar hardware [16]. Sua arquitetura separa frontend React e backend nativo Rust por IPC Tauri. O backend declara caminhos específicos para Windows, Linux e macOS, incluindo Win32 IOCTL, `lsblk/sysfs` e DiskArbitration.

A parte arquiteturalmente mais relevante é a possibilidade de customizar uma imagem ext4 sem montar o filesystem no host. Isso é particularmente valioso em Windows, onde ext4 não é nativo. Em vez de exigir uma VM Linux apenas para escrever um arquivo de configuração, o crate dedicado opera no conteúdo da partição e injeta dados de autoconfiguração.

A ferramenta também descreve streaming de `xz`, `gzip`, `zstd` e `bzip2` e verificação SHA-256. Em um fluxo de ins-

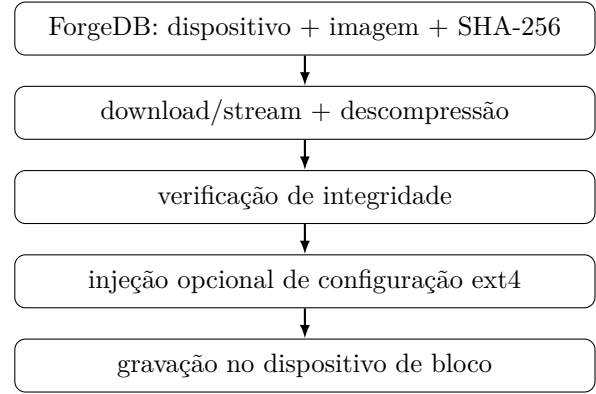


Figura 4: Fluxo lógico do FORGEIMAGER.

talação, a validação deve ocorrer antes de disponibilizar a mídia como concluída. A Figura 4 resume o caminho.

9 ForgeHub: gerenciamento em tempo de execução

9.1 Daemon e persistência

O FORGEHUB inicializa uma base bbolt em `/opt/multiforge/state/forgehub.db`, com fallback para `./data` em ambiente de desenvolvimento, registra middleware HTTP e embute assets web diretamente no executável [10]. O binário também instancia um gerenciador de reverse proxy dinâmico, permitindo que módulos sejam publicados sob uma superfície HTTP comum.

Assets imutáveis podem receber cache de um ano, enquanto o HTML principal usa `no-cache`. Essa estratégia é adequada a appliances cuja interface web deve carregar rapidamente sem manter uma cadeia de frontend separada em produção.

9.2 Runner híbrido e guarda de memória

O runner lê `MemAvailable` de `/proc/meminfo`; se a RAM disponível estiver abaixo do limiar de 300 MB ou do requisito declarado pelo módulo, a inicialização é recusada [13]. Para módulos `systemd`, a execução é confirmada com `systemctl is-active`; para módulos `compose`, o runner verifica disponibilidade do Docker e exige pelo menos um serviço em estado running.

Esse mecanismo é simples, mas importante em 2 GB: um catálogo de aplicações sem admission control pode levar o appliance a thrashing/OOM. A arquitetura desloca parte da decisão de instalação/execução para metadados de recursos.

9.3 Telemetria

A telemetria atual coleta CPU, memória, temperatura, disco e I/O de rede, mantendo um snapshot atualizado a cada segundo para atender requisições rapidamente [15]. No hardware alvo, a RAM é lida diretamente quando o total detectado está entre 1500 e 2100 MB. Em hosts de desenvolvimento, o código normaliza a memória para

um envelope simulado de 1805 MB. A temperatura usa 42,5 °C como fallback se o sensor não puder ser lido.

Essa decisão torna a interface consistente em CI, mas exige cuidado científico: métricas obtidas em host de desenvolvimento não devem ser confundidas com medição física da BTV. Da mesma forma, o runner contém uma linha de heartbeat textual que imprime “RSS: 1.8MB, thermal: OK” independentemente de uma amostragem real; essa mensagem é funcional para manter o stream vivo e **não constitui telemetria experimental** [13].

10 Modelo de módulos e extensibilidade

O manifesto documentado por FORGEMODULES contém ID, versão, categoria, requisitos mínimos, dependências de sistema, origem e comandos de ciclo de vida [17]. A combinação com FORGEDB permite que o catálogo descreva “o que existe”, enquanto o FORGEHUB decide “se cabe” e “como executar”.

O ciclo conceitual é:

descoberta → validação → admission control → instalação → start → health → stop

Esse desenho evita acoplar a imagem base a uma única aplicação. O caso M.A.B.I., uma assistente acadêmica descrita no README principal e na release v2.2.0, funciona como demonstração de que a box pode hospedar workloads de voz, interface e dados acadêmicos [21, 25]. Entretanto, como o código do módulo não está presente em **ForgeModules** no snapshot auditado, este artigo não usa o módulo como base para afirmações de desempenho do core.

11 Caso de uso institucional: appliance acadêmico

A aplicação acadêmica motiva requisitos que são mais amplos que “rodar Linux”. Um totem em espaço universitário precisa:

- inicializar sem operador técnico local;
- recuperar-se de credenciais inválidas;
- conectar-se a redes 802.1X;
- exibir estado legível em HDMI;
- continuar útil em degradação de internet;
- aceitar aplicações sem reconstruir a imagem base;
- oferecer telemetria e diagnóstico remoto.

Na release v2.2.0, o projeto registra adaptação para dados de Engenharia de Controle e Automação e Engenharia Ambiental da UNESP Sorocaba, além de suporte a eduroam [21]. Do ponto de vista arquitetural, o aspecto relevante não é a base acadêmica específica, mas a separação entre infraestrutura e aplicação: FORGEOS fornece boot, rede e recuperação; FORGEHUB fornece gestão; a aplicação consome os recursos disponíveis.

12 Otimizações: auditoria de evidência

A Tabela 3 consolida as principais otimizações e o nível de evidência encontrado.

A tabela também revela uma diferença importante entre *otimização plausível* e *otimização demonstrada*. Por exemplo, MGLRU é tecnicamente adequado a pressão de memória e é documentado pelo kernel Linux como alternativa de reclaim [31]; isso não substitui a necessidade de demonstrar que o kernel distribuído o habilita. Similarmente, o artigo original do BBR oferece fundamento para a escolha do algoritmo [2], mas não prova que a TV Box obtém maior throughput em sua rede específica.

13 Validação e resultados disponíveis

13.1 Artefato publicado

A release v2.2.0 fornece:

- `ForgeOS_BTV_E10_v2.2.0.img.xz`, 738.523.064 bytes;
- checksum SHA-256 publicado;
- `forge-images.json`;
- log do teste QEMU, com 97.293 bytes na release analisada.

Esses elementos constituem evidência de distribuição e rastreabilidade [21]. O catálogo compilado também registra versões anteriores e seus tamanhos, permitindo observar evolução do artefato ao longo do desenvolvimento [26].

13.2 Gate QEMU

O harness extrai kernel e initrd da imagem, inicializa `qemu-system-aarch64` com máquina `virt`, CPU Cortex-A53, quatro vCPUs e 1 GB de RAM, anexa a imagem via VirtIO e procura sinais de inicialização do kernel, `systemd`, `panic` e exceções Python [27]. A imagem é rejeitada se o kernel não iniciar ou se determinados erros críticos forem encontrados.

Esse teste é útil como *software gate*, mas não substitui *hardware-in-the-loop*. A máquina QEMU `virt` não reproduz o Device Tree Meson G12A, o SDIO RTL8189FTV, Ethernet ou o HDMI real. Portanto, uma imagem que passa no QEMU ainda precisa de validação na BTV.

13.3 Infraestrutura de testes

O repositório descreve uma estratégia E2E de quatro níveis para o ForgeHub: contratos isolados, condições de borda, concorrência e jornadas completas [9]. Entre os requisitos aparecem status, métricas, SSE, memória, módulos, proxy, Wi-Fi, redaction de segredos e rollback.

A documentação de testes deve ser distinguida do resultado de execução: metas como “latência < 50 ms” e “RSS ≤ 15 MB” são contratos ou alvos, não medições automaticamente aceitas como resultados. Uma versão submetida a periódico deve anexar logs por commit e ambiente, com estatística descritiva e repetição.

Tabela 3: Auditoria das otimizações no snapshot analisado. E1: aplicada pelo pipeline; E2: código runtime; E3: declarada mas não provada ativa; E4: ganho quantitativo requer experimento.

Mecanismo	Nível	Evidência no repositório	Interpretação científica
DTB BTV E10	E1	Builder copia DTB enterprise para <code>/boot/dtb/amlogic</code>	É parte explícita da imagem.
CMA 64 MB	E1	DTS/README do DTB	Reserva configurada; ganho real de RAM deve ser medido.
SDIO 25 MHz	E1	DTS + DTB injetado	Clock aplicado; estabilidade/throughput requer A/B.
RTL8189FTV anti-sleep	E1	<code>/etc/modprobe.d/8189fs.conf</code>	Parâmetros são injetados.
ZRAM ZSTD 50%	E1	<code>/etc/default/zramswap</code> gerado pelo builder	Configuração aplicada se o serviço zramswap existir/for iniciado na base.
Dirty ratios / cache pressure	E1	<code>99-forgeos-performance.conf</code>	Valores são instalados via sysctl.d.
FQ + BBR	E1/E3	sysctl seleciona bbr; Kconfig contém opção	Seleção é aplicada; disponibilidade e versão do BBR dependem do kernel base.
MGLRU	E3	Apenas <code>kernel_patches.config/docs</code> no pipeline observado	Deve ser verificado em <code>/proc/config.gz</code> e sysfs.
BORE scheduler	E3	Kconfig de intenção, sem compilação no builder atual	Não afirmar ativo sem kernel audit.
Kyber/BFQ	E3	Kconfig de intenção	Verificar <code>/sys/block/*/queue/scheduler</code> .
Preempt + HZ=300	E3	Kconfig de intenção	Verificar configuração efetiva do kernel.
ARMv8 crypto CE	E3	Kconfig de intenção	Verificar módulos/configuração e benchmark.
QEMU boot gate	E1	Executado antes da compressão final	Detecta falhas genéricas de kernel/rootfs/userspace; não emula periféricos Amlogic específicos.
Framebuffer writer único	E1/E2	release + units systemd/drop-in	Reduz concorrência no <code>/dev/fb0</code> ; medir boot/falhas.
Watchdog cliente/AP	E2	script atual com 60 s	Implementado; discrepância documental com 75 s.
Memory guard 300 MB	E2	runner Go	Implementado no daemon; depende de binary deployment.
Compressão XZ multithread	E1	<code>xz -T0 -4</code>	Afeta tempo de empacotamento, não runtime.
RAM disk no builder	E1	<code>tmpfs</code> quando RAM > 12 GB	Otimização de build host; medir ganho separado.
Spot VM efêmera	E1	launcher GCP 8-32 vCPU	Otimiza elasticidade/custo operacional do build; custo declarado precisa ser reavaliado por execução/região.

14 Protocolo experimental recomendado

Para transformar o artefato em uma contribuição experimental forte, propomos a seguinte metodologia.

14.1 Questões de pesquisa

RQ1: Qual o custo de recursos do FORGEOS em idle e sob workloads representativos?

RQ2: As otimizações de memória e DTB aumentam disponibilidade de RAM e estabilidade em relação à imagem Armbian base?

RQ3: O provisionamento e rollback funcionam de forma repetível sob falhas reais de autenticação, DHCP, gateway e energia?

RQ4: Qual o overhead do FORGEHUB e do gerenciamento modular?

RQ5: O pipeline de build produz artefatos rastreáveis e funcionalmente reproduzíveis em múltiplas execuções?

14.2 Baselines

Comparar:

1. imagem Armbian base sem alterações;
2. Armbian + DTB BTV E10;
3. FORGEOS completo;
4. opcionalmente, FORGEOS com otimizações individualmente removidas em estudo de ablação.

14.3 Métricas

Para cada condição, executar pelo menos 30 repetições quando aplicável e reportar mediana, média, desvio-padrão, p5/p95 e intervalo de confiança. As métricas mínimas são:

- tempo de boot até serviço pronto;
- RSS/PSS dos daemons e `MemAvailable`;
- atividade de swap/ZRAM, page faults e eventos OOM;
- I/O de eMMC/MicroSD e bytes escritos;
- throughput, RTT, perda e estabilidade Wi-Fi;
- temperatura e frequência de CPU;

- energia na tomada durante idle, boot e workload;
- tempo até recuperação AP após falha;
- sucesso de 100 ciclos de boot e 100 ciclos de provisionamento;
- uptime de 24/48/72 h;
- latência das APIs e SSE sob concorrência;
- tempo e custo do build local versus Spot VM.

14.4 Estudo de ablação

A contribuição das otimizações deve ser isolada. Um experimento recomendado é:

$$B_0 = \text{base}, \quad B_1 = B_0 + \text{DTB}, \quad B_2 = B_1 + \text{ZRAM}, \quad B_3 = B_2 + \text{sysctl}, \quad B_4 = B_3 + \text{runtime Multi-Forge}$$

Para o SDIO, testar pelo menos 25 MHz contra a configuração original, mantendo distância, AP, canal e carga constantes. Para CMA, medir `MemTotal`, `MemAvailable`, estabilidade do framebuffer e workloads gráficos. Para BBR, usar emulação de perda/RTT com `tc netem` e também rede Wi-Fi real.

15 Segurança e endurecimento

A imagem atual foi produzida em contexto de hackathon e bancada. O builder habilita `PermitRootLogin yes` e define uma senha padrão para usuários de desenvolvimento [18]. Isso é incompatível com deployment institucional sem etapa de first-boot credential rotation.

O daemon ForgeHub também configura CORS amplamente permissivo no ponto de entrada atual [10]. Embora simplifique acesso durante desenvolvimento, uma instalação em rede compartilhada deve:

- desabilitar senhas padrão e login root por senha;
- gerar chaves/credenciais únicas por dispositivo;
- restringir endpoints administrativos por autenticação;
- restringir CORS a origens conhecidas;
- separar APIs de leitura e mutação;
- proteger segredos EAP em repouso;
- aplicar least privilege às units systemd;
- assinar ou atestar imagens e catálogos;
- verificar checksum antes do flash;
- manter perfil `dev` separado de perfil `production`.

Outro ponto é a recuperação MaskROM/USB declarada pelo catálogo. Ela é útil para manutenção, mas políticas de instalação devem considerar risco físico e integridade da cadeia de boot.

16 Reprodutibilidade e cadeia de suprimentos

O projeto já possui elementos favoráveis à reprodutibilidade: configuração versionada, SHA-256, catálogo compilado, base de imagem nomeada, scripts automatizados e release com logs. Entretanto, reprodutibilidade científica forte requer mais.

Primeiro, a URL base aponta para um release externo do repositório Ophub; um fallback pode selecionar “release mais recente”, o que torna builds posteriores potencialmente diferentes mesmo sob o mesmo commit [18].

Para uma release científica, toda dependência deve ser fixada por digest.

Segundo, o builder copia binários de ForgeHub/bin apenas se eles existirem, mas esse diretório não aparece no snapshot `main` auditado. Um clone limpo pode, portanto, produzir imagem diferente de um workspace que já contenha binários. O pipeline deve construir o ForgeHub a partir de fonte como etapa obrigatória ou falhar explicitamente.

Terceiro, o diretório `ForgeModules` do snapshot contém somente documentação, embora o README principal e documentação de módulos descrevam aplicações adicionais. Para um artifact evaluation externo, os módulos necessários à demonstração devem ser submódulos fixados, artefatos versionados ou fontes presentes.

Quarto, timestamps e conteúdo de APT tornam improvável um build bit-a-bit idêntico sem snapshot de repositório e normalização de ambiente. Trabalhos sobre reproducíveis builds mostram que automação, embora necessária, não é suficiente para equivalência binária [30]. Como melhoria, recomenda-se:

- fixar imagem base por SHA-256;
- usar snapshots datados do Debian/Armbian;
- versionar lockfiles e toolchains;
- construir binários ausentes de maneira determinística;
- registrar SBOM;
- registrar `SOURCE_DATE_EPOCH`;
- armazenar manifestos de build e hashes de todos os inputs;
- executar rebuild independente em CI.

17 Sustentabilidade e computação de borda

O argumento ambiental da MULTI-FORGE deve ser apresentado com cautela. Reutilizar um dispositivo potencialmente adia descarte e evita aquisição imediata de hardware novo, mas o benefício ambiental total depende do consumo de energia, vida útil adicional e alternativa comparada. O Global E-waste Monitor mostra a escala do problema mundial, mas não mede especificamente TV Boxes brasileiras [29]. Portanto, o projeto demonstra uma *estratégia de extensão de vida útil*, não uma redução quantificada de emissões.

Do ponto de vista computacional, a BTV E10 ocupa uma faixa intermediária: mais capaz que microcontroladores tipicamente associados a TinyML, porém muito mais restrita que servidores de borda x86. A literatura de TinyML e edge intelligence destaca justamente desafios de memória, heterogeneidade e eficiência em dispositivos limitados [35]. A MULTI-FORGE ataca outra parte do problema: lifecycle, imagem, rede, deployment e gestão de aplicações em hardware reaproveitado.

18 Trabalhos relacionados

18.1 Edge e fog computing

Shi et al. definem edge computing como processamento próximo às fontes de dados, destacando latência, bandwidth e privacidade [36]. Bonomi et al. descrevem

fog computing como extensão da nuvem até a borda [1], enquanto Mao et al. sistematizam o campo de mobile edge computing e seus desafios de recursos e deployment [33]. A MULTI-FORGE não propõe um novo modelo de offloading; sua contribuição está na engenharia de plataforma necessária para transformar hardware de consumo heterogêneo em appliance gerenciável.

18.2 Deployment declarativo e builds

Nix demonstrou os benefícios de descrição rigorosa e isolamento de dependências em deployment de software [3]. Build Systems à la Carte oferece uma taxonomia para entender pipelines de build como composição de mecanismos [34]. A MULTI-FORGE é menos geral: seu catálogo declarativo descreve hardware e imagens, enquanto scripts imperativos montam a distribuição. Essa combinação é pragmática, mas cria espaço para evoluir em direção a builds herméticos.

18.3 Reprodutibilidade

Lamb e Zacchiroli discutem reproducibile builds como mecanismo para aumentar confiança entre fonte e binário [30]. A MULTI-FORGE possui checksums e automação, porém ainda não demonstra equivalência bit-a-bit. A cadeia atual é melhor descrita como *traceable build pipeline*.

18.4 Computação em dispositivos restritos

Ray revisa TinyML e identifica memória, processamento, heterogeneidade e energia como restrições centrais [35]. Embora a BTV E10 disponha de Linux completo e gigabytes de RAM, a preocupação com footprint permanece relevante. ZRAM, admission control de módulos, daemon único e UI framebuffer são exemplos de decisões motivadas por essa limitação.

19 Ameaças à validade

Validade interna. O projeto evolui rapidamente. Documentação e código apresentam discrepâncias, como timeout de watchdog e presença de módulos. Por isso, todas as conclusões estão vinculadas ao commit auditado.

Validade de construção. “Otimização” não implica melhoria observada. Diversas configurações têm racional técnico, mas ainda carecem de A/B tests. O artigo explicitamente não transforma configuração em ganho percentual.

Validade externa. A avaliação de hardware está centrada em um único modelo, BTV E10/S905X2. Não há base experimental para generalizar compatibilidade a TV Boxes Amlogic, Rockchip ou Qualcomm em geral.

Validade de reprodutibilidade. Dependências externas e binários opcionais podem fazer builds de workspaces diferentes divergir. Isso deve ser corrigido antes de reivindicar build hermético.

Validade operacional. QEMU usa a máquina genérica *virt*, portanto não valida o conjunto real de pe-

riféricos, sinal elétrico SDIO, HDMI ou watchdog do SoC.

Segurança. O perfil de desenvolvimento contém credenciais e permissões que não devem ser extrapoladas para produção.

20 Discussão

A principal lição arquitetural é que reaproveitar hardware não é o mesmo que simplesmente fazê-lo inicializar Linux. Um appliance de borda exige pelo menos cinco propriedades: identificação precisa do dispositivo, imagem rastreável, bootstrap que dispense periféricos, mecanismo de recuperação e runtime com limites explícitos de recursos. A MULTI-FORGE implementa ou especifica esses elementos em componentes separados.

A decisão de manter FORGEDB independente do FORGEIMAGER reduz acoplamento e permite evolução do catálogo. A decisão de usar um daemon Go único para gestão diminui a quantidade de processos de controle no runtime. O uso de framebuffer direto no caminho crítico evita compositor gráfico durante onboarding. A guarda de memória previne que a extensibilidade modular torne o sistema autodestrutivo sob 2 GB.

Por outro lado, a auditoria mostra que o projeto ainda contém sinais naturais de desenvolvimento acelerado: documentação de gerações antigas, parâmetros de kernel não integrados ao build, timeouts divergentes e artefatos esperados mas ausentes do clone. Transformar um protótipo de hackathon em plataforma de pesquisa requer exatamente o trabalho menos glamoroso: fixar versões, apagar credenciais de bancada, instrumentar experimentos e fazer o pipeline falhar quando um input obrigatório faltar.

21 Trabalhos futuros

Os próximos passos técnicos prioritários são:

1. integrar uma etapa explícita de kernel audit/build ou remover do release qualquer opção Kconfig não verificada;
2. criar perfis **dev** e **production**;
3. tornar o build hermético, com imagem base e pacotes fixados por digest;
4. incluir build obrigatório do ForgeHub no pipeline;
5. restaurar ou versionar os módulos usados pela demonstração;
6. executar o protocolo experimental da Seção anterior;
7. adicionar hardware-in-the-loop no CI, quando possível;
8. medir consumo energético e vida útil para sustentar a contribuição ambiental;
9. homologar um segundo dispositivo ARM para testar generalização do modelo declarativo;
10. produzir SBOM e assinatura/atestado de release.

22 Conclusão

Este trabalho apresentou a MULTI-FORGE, uma arquitetura de reaproveitamento de TV Boxes ARM64 como

appliances de computação de borda. A contribuição vai além de uma imagem Linux customizada: o sistema combina catálogo declarativo, gravador multiplataforma, pipeline de imagem, Device Tree específico, bootstrap de rede offline-first, recuperação automática, gerenciamento de aplicações e telemetria.

A BTV Express E10 serve como estudo de caso concreto para explorar as tensões de sistemas embarcados Linux com apenas 2 GB de RAM e armazenamento flash limitado. O ForgeOS aplica uma série de mecanismos diretamente verificáveis no pipeline, como DTB customizado, ZRAM, ajustes de `sysctl`, parâmetros do RTL8189FTV, controle de serviços e validação QEMU. Ao mesmo tempo, a auditoria identificou otimizações de kernel apenas declaradas, e não demonstradas pelo builder atual. Essa separação é importante: rigor científico exige distinguir intenção, implementação e efeito mensurado.

A release v2.2.0 e sua cadeia de artifacts demonstram maturidade suficiente para caracterização como protótipo funcional de sistemas. Para uma contribuição experimental de maior alcance, faltam medições controladas de boot, memória, rede, energia, estabilidade e ablação das otimizações. Com essas etapas e o endurecimento da reprodutibilidade, a MULTI-FORGE pode funcionar não apenas como solução de hackathon, mas como plataforma experimental para investigar reutilização de hardware ARM de consumo em edge computing.

Disponibilidade do artefato

O código-fonte está disponível publicamente em:

<https://github.com/multi-forge/multi-forge>

Snapshot utilizado neste manuscrito:

f670eb822fb40d6fda17683496c85b04b2711dec

A release analisada é FORGEOS v2.2.0. Para resultados reproduzíveis, recomenda-se citar simultaneamente commit, tag de release, SHA-256 da imagem e versão do hardware/PCB.

Referências

- [1] Flavio Bonomi, Rodolfo Milito, Jiang Zhu, and Sateesh Addepalli. Fog computing and its role in the internet of things. In *Proceedings of the First Edition of the MCC Workshop on Mobile Cloud Computing*, pages 13–16. ACM, 2012.
- [2] Neal Cardwell, Yuchung Cheng, C. Stephen Gunn, Soheil Hassas Yeganeh, and Van Jacobson. Bbr: Congestion-based congestion control. *ACM Queue*, 14(5):20–53, 2016.
- [3] Eelco Dolstra, Merijn de Jonge, and Eelco Visser. Nix: A safe and policy-free system for software deployment. In *18th Large Installation System Administration Conference (LISA 04)*, pages 79–92. USENIX Association, 2004.
- [4] Equipe Multi-Forge. Btv e10 enterprise device tree. <https://github.com/multi-forge/multi-forge/tree/main/ForgeCore/dtb>, 2026.
- [5] Equipe Multi-Forge. Forge ap controller. <https://github.com/multi-forge/multi-forge/blob/main/ForgeHub/hardware/network/forge-ap-ctrl>, 2026.
- [6] Equipe Multi-Forge. Forgecore. <https://github.com/multi-forge/multi-forge/tree/main/ForgeCore>, 2026.
- [7] Equipe Multi-Forge. Forgedb btv express e10 device descriptor. <https://github.com/multi-forge/multi-forge/blob/main/ForgeDB/devices/btv/e10/device.yaml>, 2026.
- [8] Equipe Multi-Forge. Forgedb declarative hardware and image catalog. <https://github.com/multi-forge/multi-forge/tree/main/ForgeDB>, 2026.
- [9] Equipe Multi-Forge. Forgehub 4-tier opaque-box e2e testing infrastructure. https://github.com/multi-forge/multi-forge/blob/main/ForgeHub/TEST_INFRA.md, 2026.
- [10] Equipe Multi-Forge. Forgehub daemon entry point. <https://github.com/multi-forge/multi-forge/blob/main/ForgeHub/backend/cmd/forgehub/main.go>, 2026.
- [11] Equipe Multi-Forge. Forgehub framebuffer kiosk systemd unit. <https://github.com/multi-forge/multi-forge/blob/main/ForgeHub/hardware/systemd/forge-kiosk.service>, 2026.
- [12] Equipe Multi-Forge. Forgehub hardware connectivity watchdog. <https://github.com/multi-forge/multi-forge/blob/main/ForgeHub/hardware/network/watchdog.sh>, 2026.
- [13] Equipe Multi-Forge. Forgehub hybrid module runner. <https://github.com/multi-forge/multi-forge/blob/main/ForgeHub/backend/internal/runner/runner.go>, 2026.
- [14] Equipe Multi-Forge. Forgehub minimal panels systemd drop-in. <https://github.com/multi-forge/multi-forge/blob/main/ForgeHub/hardware/systemd/50-minimal-panels.conf>, 2026.
- [15] Equipe Multi-Forge. Forgehub telemetry subsystem. <https://github.com/multi-forge/multi-forge/blob/main/ForgeHub/backend/internal/telemetry/telemetry.go>, 2026.
- [16] Equipe Multi-Forge. Forgeimager. <https://github.com/multi-forge/multi-forge/tree/main/ForgeImager>, 2026.
- [17] Equipe Multi-Forge. Forgemodules manifest and lifecycle specification. <https://github.com/multi-forge/multi-forge/blob/main/ForgeModules/README.md>, 2026.

- [18] Equipe Multi-Forge. Forgeos image builder. <https://github.com/multi-forge/multi-forge/blob/main/ForgeCore/builder/build-image.sh>, 2026.
- [19] Equipe Multi-Forge. Forgeos kernel patch configuration targets. https://github.com/multi-forge/multi-forge/blob/main/ForgeCore/builder/config/kernel_patches.config, 2026.
- [20] Equipe Multi-Forge. Forgeos performance and stability sysctl configuration. <https://github.com/multi-forge/multi-forge/blob/main/ForgeCore/builder/config/sysctl-performance.conf>, 2026.
- [21] Equipe Multi-Forge. Forgeos v2.2.0 – production release. <https://github.com/multi-forge/multi-forge/releases/tag/v2.2.0>, 2026. Publicada em 17 set. 2026.
- [22] Equipe Multi-Forge. Forgeprovisioner. <https://github.com/multi-forge/multi-forge/tree/main/ForgeProvisioner>, 2026.
- [23] Equipe Multi-Forge. Google cloud spot image builder launcher. <https://github.com/multi-forge/multi-forge/blob/main/ForgeCore/builder/gcp-spot-launcher.py>, 2026.
- [24] Equipe Multi-Forge. Legacy forgeprovisioner connectivity watchdog. <https://github.com/multi-forge/multi-forge/blob/main/ForgeProvisioner/bin/watchdog.sh>, 2026.
- [25] Equipe Multi-Forge. Multi-forge / m.a.b.i. – repositório principal. <https://github.com/multi-forge/multi-forge>, 2026. Snapshot audiado em 17 set. 2026.
- [26] Equipe Multi-Forge. Multi-forge source snapshot. <https://github.com/multi-forge/multi-forge/commit/f670eb822fb40d6fda17683496c85b04b2711dec>, 2026. Commit f670eb822fb40d6fda17683496c85b04b2711dec.
- [27] Equipe Multi-Forge. Qemu aarch64 virtual boot verification harness. <https://github.com/multi-forge/multi-forge/blob/main/ForgeCore/builder/qemu-verify-boot.sh>, 2026.
- [28] Equipe Multi-Forge. Rtl8189ftv module options. <https://github.com/multi-forge/multi-forge/blob/main/ForgeCore/builder/config/8189fs-performance.conf>, 2026.
- [29] International Telecommunication Union and UNITAR. The global e-waste monitor 2024. Technical report, ITU and UNITAR, 2024.
- [30] Chris Lamb and Stefano Zacchiroli. Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software*, 39(2):62–70, 2022.
- [31] Linux Kernel Documentation. Multi-gen lru. https://docs.kernel.org/mm/multigen_lru.html, 2026. Acesso em 17 set. 2026.
- [32] Linux Kernel Documentation. zswap. <https://www.kernel.org/doc/html/next/admin-guide/mm/zswap.html>, 2026. Referência conceitual para memória comprimida; ForgeOS usa ZRAM, não zswap.
- [33] Yuyi Mao, Changsheng You, Jun Zhang, Kaibin Huang, and Khaled B. Letaief. A survey on mobile edge computing: The communication perspective. *IEEE Communications Surveys & Tutorials*, 19(4):2322–2358, 2017.
- [34] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build systems à la carte. *Proceedings of the ACM on Programming Languages*, 2(ICFP):1–29, 2018.
- [35] Partha Pratim Ray. A review on tinymt: State-of-the-art and prospects. *Journal of King Saud University - Computer and Information Sciences*, 34(4):1595–1623, 2022.
- [36] Weisong Shi, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu. Edge computing: Vision and challenges. *IEEE Internet of Things Journal*, 3(5):637–646, 2016.

A Checklist de reprodução

1. Clonar o repositório no commit citado.
2. Verificar a imagem base e registrar seu SHA-256.
3. Executar o builder em Linux/WSL ou VM Spot.
4. Preservar `qemu-boot-test.log`.
5. Validar SHA-256 da imagem produzida.
6. Gravar MicroSD via ForgeImager ou dd.
7. Registrar revisão física da BTV E10.
8. Capturar `uname -a`, `/proc/config.gz`, `/proc/meminfo`, `swapon --show`, `sysctl` e `schedulers` ativos.
9. Executar testes de boot/provisionamento repetidos.
10. Arquivar logs brutos e scripts de análise.

B Comandos mínimos de auditoria no dispositivo

```
uname -a
cat /proc/device-tree/model
zcat /proc/config.gz | grep -E \
'LRU_GEN|SCHED_BORE|TCP_CONG_BBR|BFQ|KYBER'
swapon --show
zramctl
sysctl vm.swappiness vm.dirty_ratio \
net.ipv4.tcp_congestion_control \
net.core.default_qdisc
cat /sys/block/mmcblk*/queue/scheduler
cat /sys/class/thermal/thermal_zone0/temp
free -m
systemctl status forgehub forge-kiosk \
forge-ap forge-watchdog
```

C Evidências primárias do repositório

Os principais arquivos utilizados para reconstruir a arquitetura são:

- README.md;
- ForgeCore/builder/build-image.sh;
- ForgeCore/builder/qemu-verify-boot.sh;
- ForgeCore/builder/gcp-spot-launcher.py;
- ForgeCore/builder/config/sysctl-performance.conf;
- ForgeCore/builder/config/8189fs-performance.conf;
- ForgeCore/builder/config/kernel_patches.config;
- ForgeCore/dtb/meson-g12a-btv-e10-enterprise.dts;
- ForgeDB/devices/btv/e10/device.yaml;
- ForgeDB/README.md;
- ForgeImager/README.md;
- ForgeProvisioner/README.md;
- ForgeProvisioner/bin/watchdog.sh;
- ForgeHub/backend/cmd/forgelhub/main.go;
- ForgeHub/backend/internal/runner/runner.go;
- ForgeHub/backend/internal/telemetry/telemetry.go;
- ForgeHub/hardware/network/forgelap-ctrl;
- ForgeHub/hardware/network/watchdog.sh;
- ForgeHub/hardware/systemd/forgelkiosk.service;
- ForgeModules/README.md;
- release GitHub v2.2.0.